

Case Studies I

The method applied end-to-end — the first five real systems, each from requirements to trade-offs.

55 Design a URL Shortener — unique keys, read-heavy, redirects

56 Design Pastebin — storing large text blobs, expiry, CDN

57 Design a News Feed (Twitter) — the fan-out problem

58 Design a Chat System (WhatsApp) — real-time delivery & presence

59 Design a Notification System — multi-channel fan-out at scale

Each case study uses the framework from Chapter 52: requirements → estimation → API → high-level design → deep dive → trade-offs.

CHAPTER 55

Design a URL Shortener

Part X · Case Studies — unique keys, a read-heavy load, and fast redirects.

WHAT THIS CASE STUDY SHOWS

- Generating short, unique keys at scale.
- Designing for an extremely read-heavy workload.
- Applying caching, CDN, and sharding to a simple-looking problem.

THE CORE CHALLENGE

A URL shortener looks trivial but hides two real problems: **generating unique short keys** without collisions at scale, and serving **billions of redirects** with minimal latency. It's the perfect first case study — small surface, real depth.

55.1 Requirements

Functional: shorten a long URL to a short one; redirect a short URL to the original; optional custom aliases and expiry. **Non-functional:** very low redirect latency, high availability (a broken redirect is a broken link), and massive read scale. Reads vastly outnumber writes.

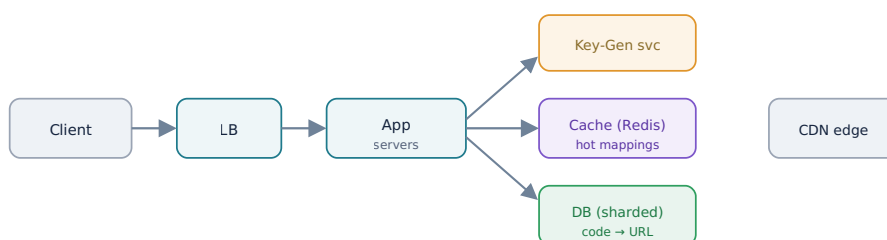
55.2 Estimation

Say 100M new URLs/day → ~1,000 write QPS ($\div \sim 100k$ s), and reads at $\sim 10\times \rightarrow \sim 10,000$ read QPS, higher at peak (Chapter 53). Over 5 years at ~500 bytes each → roughly tens of TB — large but not enormous. The read:write skew (10:1+) dominates the design.

55.3 API

Endpoint	Purpose
<code>POST /urls {long_url}</code>	Create a short URL → returns short code.
<code>GET /{short_code}</code>	Redirect (HTTP 301/302) to the long URL.

55.4 High-level design



writes take a key from the generator; reads hit cache/CDN first, then the sharded DB

55.5 Deep dive: generating short keys

KEY INSIGHT — ENCODE A COUNTER, DON'T HASH AND PRAY

Two approaches. **Hashing** the long URL (e.g. MD5) and taking a prefix is simple but risks **collisions** you must detect and resolve. Cleaner: keep a **global counter** and **Base62-encode** it (a-z, A-Z, 0-9) — 7 characters give $62^7 \approx 3.5$ trillion unique codes, collision-free by construction. To avoid a single-counter bottleneck, a **key-generation service** hands out ranges of IDs to app servers (or uses a distributed ID scheme). Reads are then a simple key lookup — perfect for caching.

55.6 Scaling reads and trade-offs

Because it's read-heavy, put a **cache** (Redis, Chapter 8) in front of the database for hot codes, and let a **CDN**/edge handle popular redirects. **Shard** the mapping table by short code (Chapter 17). Choose **301** (permanent, more cacheable) vs **302** (temporary, lets you count clicks) — a caching-vs-analytics trade-off. Use eventual consistency for click counts (Chapter 21).

Key takeaways

- **Base62-encode a counter** for collision-free short keys; a key-gen service avoids the counter bottleneck.
- The workload is **read-heavy** — cache and CDN carry most traffic; shard the store.
- Pick 301 vs 302 as a **cacheability vs analytics** trade-off; counts can be eventually consistent.

CHAPTER 56

Design Pastebin

Part X · Case Studies — storing large text blobs with expiry and a CDN.

WHAT THIS CASE STUDY SHOWS

- Separating metadata from large content (blob storage).
- Handling expiration and read-heavy delivery.
- Extending the URL-shortener pattern to bigger payloads.

THE CORE CHALLENGE

Pastebin is a URL shortener whose “value” is a **large text blob**, not a short string. The new problem: don’t stuff big content into your primary database — store it where large objects belong, and serve it cheaply.

56.1 Requirements

Functional: create a paste (text) and get a short URL; view a paste; optional **expiration** and visibility. **Non-functional:** read-heavy, low-latency reads, durable storage, and reasonable write throughput.

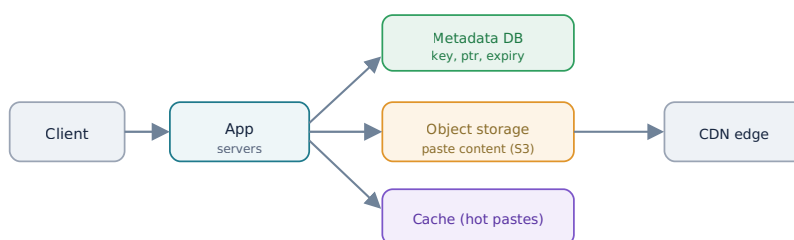
56.2 Estimation

Say 1M new pastes/day (~10 write QPS) with reads ~10x. Pastes are larger — average tens of KB — so **storage and bandwidth** dominate more than in the URL case. Model both content size and retention (Chapter 53).

56.3 API

Endpoint	Purpose
POST /pastes {content, expiry}	Store content, return a short key.
GET /{key}	Fetch the paste content.

56.4 High-level design



small metadata in the DB; large content in object storage; hot reads via cache/CDN

56.5 Deep dive: metadata vs content, and expiry

KEY INSIGHT — STORE BIG BLOBS WHERE BLOBS BELONG

Keep only small **metadata** (key, a pointer, size, expiry) in the database, and put the actual **content in object storage** (S3-style) that's built for large, cheap, durable blobs and integrates with a CDN. This keeps the database small and fast and slashes cost. For **expiration**, store an expiry timestamp and lazily treat expired pastes as gone (return 404), with a background job reclaiming space — simpler and cheaper than deleting exactly on time.

56.6 Trade-offs

Object storage adds a second lookup (metadata → content) but keeps the primary DB lean and lets the **CDN** serve popular pastes directly (Chapter 8). Choose whether keys are random (unguessable, more private) or sequential (Chapter 55). Content is immutable, so caching is easy and safe.

Key takeaways

- Split **metadata (DB)** from **content (object storage)** — the defining move for blob-heavy systems.
- Serve hot, immutable content via **cache and CDN**.
- Handle **expiry lazily** with a reclaiming background job.

CHAPTER 57

Design a News Feed (Twitter)

Part X · Case Studies — the fan-out problem.

WHAT THIS CASE STUDY SHOWS

- Fan-out on write vs fan-out on read.
- The celebrity problem and the hybrid solution.
- Building feeds with caches and queues at scale.

THE CORE CHALLENGE

When someone opens their feed, how do you assemble the latest posts from everyone they follow — fast, for hundreds of millions of users? The answer hinges on **when** you do the work: at write time or at read time. This is the classic **fan-out** problem.

57.1 Requirements

Functional: post a message; follow users; view a home feed of followed users' recent posts (roughly reverse-chronological). **Non-functional:** very low feed-load latency, extreme read scale, high availability; eventual consistency is fine (a post appearing a second late is OK).

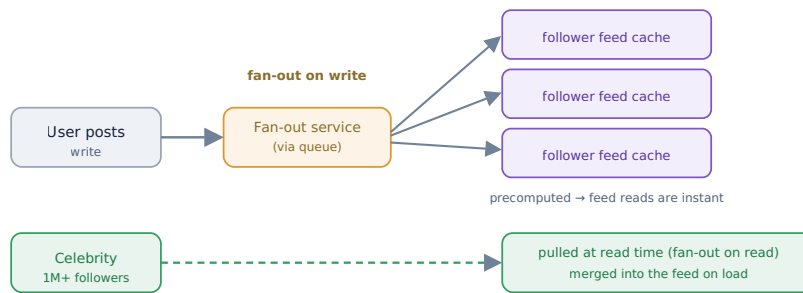
57.2 Estimation

Hundreds of millions of DAU, each loading the feed many times → feed reads are the dominant operation (hundreds of thousands of QPS at peak). Writes (posts) are far fewer but **amplify**: one post by a user with a million followers may touch a million feeds. That amplification is the crux (Chapter 53).

57.3 API

Endpoint	Purpose
POST /posts {text}	Publish a post.
POST /follow {user_id}	Follow a user.
GET /feed	Get the home timeline (paginated).

57.4 Deep dive: fan-out on write vs read



KEY INSIGHT — THE HYBRID BEATS EITHER PURE APPROACH

Fan-out on write (push): when you post, write the post into each follower’s precomputed feed (in a cache) via a queue. Feed **reads** are then instant — but a **celebrity** with millions of followers causes a huge write storm. **Fan-out on read** (pull): store posts once and assemble the feed at read time — cheap writes, but expensive, slow reads. The production answer is a **hybrid**: fan-out on write for normal users, and for a few celebrities, **pull** their posts at read time and merge them in. This tames the celebrity problem while keeping most reads fast.

57.5 Trade-offs

Precomputed feeds trade **storage and write amplification** for fast reads (usually worth it, since feeds are read far more than written). Feeds are **eventually consistent** (Chapter 21). Use a queue (Chapter 28) to absorb fan-out spikes, cache feeds in Redis (Chapter 8), and shard by user (Chapter 17).

Key takeaways

- The design decision is **when** to do the work: fan-out on **write** (fast reads) vs **read** (cheap writes).
- Use a **hybrid**: push for normal users, pull for celebrities — solving the celebrity write storm.
- Precompute feeds in a **cache**, fan out via a **queue**, accept **eventual consistency**.

CHAPTER 58

Design a Chat System (WhatsApp)

Part X · Case Studies — real-time delivery and presence.

WHAT THIS CASE STUDY SHOWS

- Real-time, bidirectional messaging with persistent connections.
- Delivering to online and offline users.
- Presence, delivery receipts, and connection management at scale.

THE CORE CHALLENGE

Chat needs **instant, two-way delivery** and must handle recipients who are offline. That rules out request-response polling and centers the design on **persistent connections** plus reliable message routing and storage.

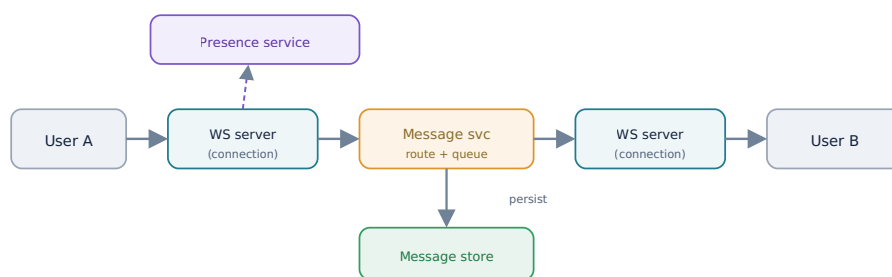
58.1 Requirements

Functional: 1:1 (and group) messaging, delivery/read receipts, online **presence**, message history, offline delivery. **Non-functional:** low latency, reliable ordered delivery, high availability, and huge numbers of concurrent connections.

58.2 Estimation

Hundreds of millions of users, tens of billions of messages/day, and — the hard part — **hundreds of millions of simultaneous open connections**. Messages are small; the challenge is connection scale and delivery, not storage volume (Chapter 53).

58.3 High-level design



persistent WebSocket connections; the message service routes and persists; presence tracks who's online

58.4 Deep dive: delivery and connections

KEY INSIGHT — THE CONNECTION IS THE HARD PART

Each client holds a persistent **WebSocket** (Chapter 31) to a connection server, which is **stateful** — so you need a registry mapping *user* → *which server holds their connection* and a **message queue/bus** so a message arriving on one server reaches the recipient's server (the backplane idea from Chapter 31). If the recipient is **online**, push instantly; if **offline**,

persist the message and deliver on reconnect (and send a push notification). Every message is **stored** for history and receipts. Ordering per conversation and idempotent delivery (Chapter 28) prevent duplicates and scrambling.

58.5 Trade-offs

Persistent connections cost memory per user and demand careful **connection management** and sticky routing. **Presence** at scale is expensive, so it's often approximate/ eventually consistent. Group chat multiplies fan-out (like Chapter 57). End-to-end encryption (Chapter 43) adds privacy at the cost of server-side features.

Key takeaways

- Chat is built on **persistent WebSocket connections** plus a **message bus** to route across connection servers.
- **Online** → push instantly; **offline** → persist and deliver on reconnect + push notification.
- Store every message; ensure **per-conversation ordering** and idempotent delivery.
- Presence and group fan-out are the expensive parts — often approximate.

CHAPTER 59

Design a Notification System

Part X · Case Studies — multi-channel fan-out at scale.

WHAT THIS CASE STUDY SHOWS

- Delivering across multiple channels (push, email, SMS).
- Decoupling with queues and workers for reliability and scale.
- Handling third-party gateways, rate limits, and de-duplication.

THE CORE CHALLENGE

A notification system must reliably deliver huge volumes of messages across **different channels** — each with its own external gateway, failure modes, and rate limits — without letting a slow or failing channel block the rest. It's a showcase for queues, workers, and resilience.

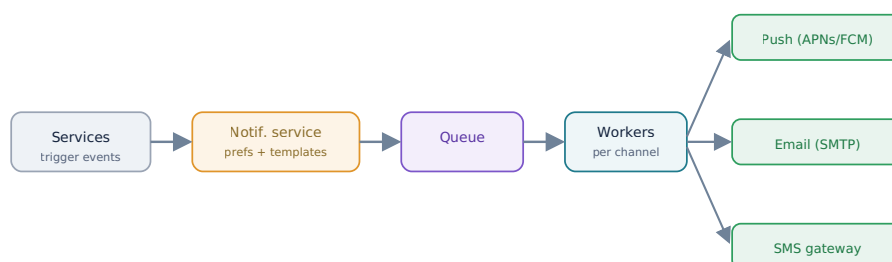
59.1 Requirements

Functional: send notifications via **push, email, and SMS**; support templates; respect user preferences/opt-outs. **Non-functional:** high throughput, reliable at-least-once delivery, resilience to third-party outages, and no duplicate spam.

59.2 Estimation

Potentially **billions** of notifications/day across channels, extremely spiky (a broadcast to all users). The system must absorb spikes and deliver steadily — a load-leveling problem (Chapters 5, 28).

59.3 High-level design



a queue decouples producers from per-channel workers that call external gateways

59.4 Deep dive: queues, channels, and reliability

KEY INSIGHT — A QUEUE PER CHANNEL DECOUPLES AND PROTECTS

Producers just publish a “notify” request; a **queue** (Chapter 28) buffers it, and **per-channel workers** pull and call the right external gateway (APNs/FCM for push, SMTP for email, an SMS provider). This **decouples** senders from slow gateways, absorbs spikes, and lets each

channel scale and fail independently (a bulkhead, Chapter 37). External calls get **retries with backoff** (Chapter 36), a **circuit breaker** when a provider is down, and a **dead-letter queue** for messages that keep failing. **Idempotency keys** and de-duplication prevent double-sending under at-least-once delivery, and **rate limiting** (Chapter 35) respects both provider limits and the user (no notification spam).

59.5 Trade-offs

At-least-once delivery plus de-duplication is usually the right balance (exactly-once is costly, Chapter 28). Priority queues let time-sensitive notifications (a security code) jump ahead of bulk marketing. Respecting user preferences and quiet hours matters as much as raw delivery. Track delivery status for observability (Chapter 50).

Key takeaways

- Decouple with a **queue**; use **per-channel workers** that call external gateways independently.
- Wrap external calls in **retries, backoff, circuit breakers, and dead-letter queues**.
- Use **idempotency + de-duplication** and **rate limiting** to avoid duplicates and spam.
- Prioritize urgent notifications; respect user preferences.

COMING NEXT

Chapters 60-64 continue the case studies: a rate limiter, a web crawler, search autocomplete, a video-streaming platform (YouTube), and photo sharing (Instagram).